Podishetti Eshwar

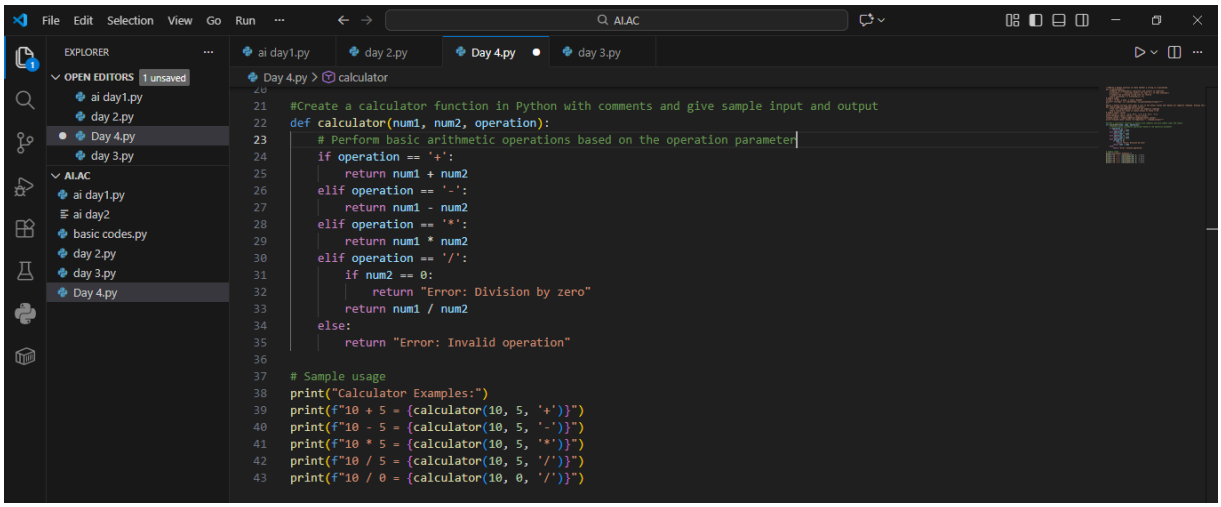
2403A51L26

B-51

ASSIGNMENT -3.2

Task 1: Progressive Prompting – Calculator Design

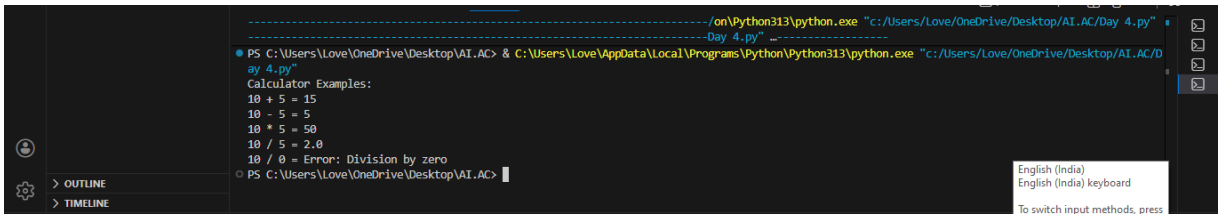
PROMPT: Create a calculator function in Python with comments and give sample input and output.



```python
#Create a calculator function in Python with comments and give sample input and output
def calculator(num1, num2, operation):
    # Perform basic arithmetic operations based on the operation parameter
    if operation == '+':
        return num1 + num2
    elif operation == '-':
        return num1 - num2
    elif operation == '*':
        return num1 * num2
    elif operation == '/':
        if num2 == 0:
            return "Error: Division by zero"
        return num1 / num2
    else:
        return "Error: Invalid operation"

# Sample usage
print("Calculator Examples:")
print(f"10 + 5 = {calculator(10, 5, '+')}")
print(f"10 - 5 = {calculator(10, 5, '-')}")
print(f"10 * 5 = {calculator(10, 5, '*')}")
print(f"10 / 5 = {calculator(10, 5, '/')}")
print(f"10 / 0 = {calculator(10, 0, '/')}")
```

OUTPUT:



```
PS C:\Users\Love\OneDrive\Desktop\AI.AC> & C:\Users\Love\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Love/OneDrive/Desktop/AI.AC/Day 4.py"
Calculator Examples:
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2.0
10 / 0 = Error: Division by zero
PS C:\Users\Love\OneDrive\Desktop\AI.AC>
```

EXPLANATION:

When we give only a function name, the AI generates very basic or incomplete code.

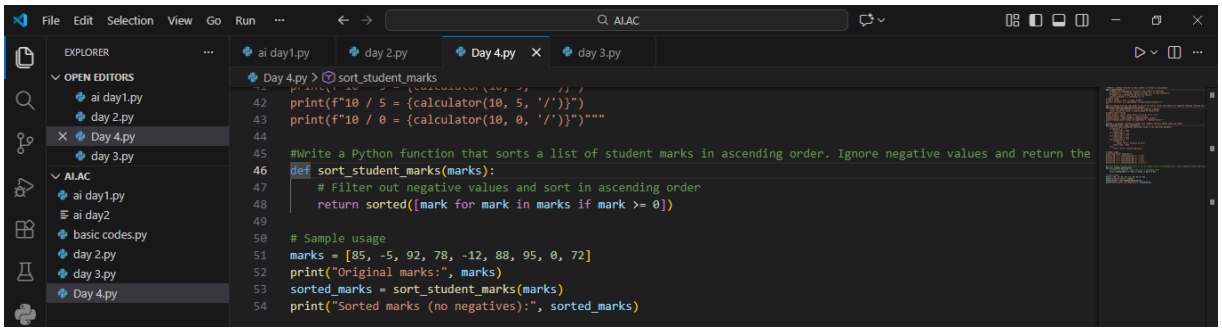
As we gradually add comments, requirements, and examples, the AI understands better and produces:

- Proper logic ,Error handling , Cleaner structure

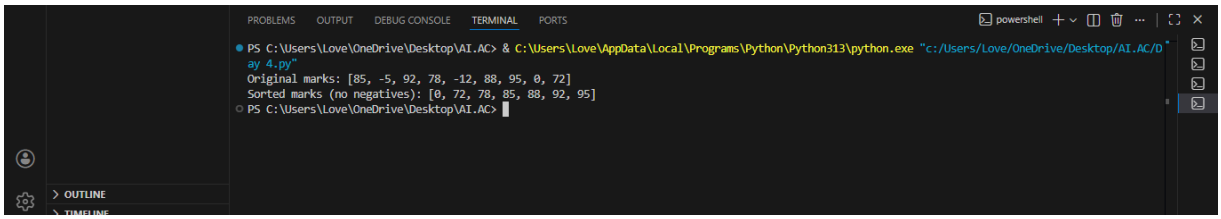
This shows that well-defined prompts lead to better AI-generated programs.

Task 2: Refining Prompts – Sorting Student Marks

PROMPT: Write a Python function that sorts a list of student marks in ascending order. Ignore negative values and return the sorted list using efficient logic.



OUTPUT:



EXPLANATION:

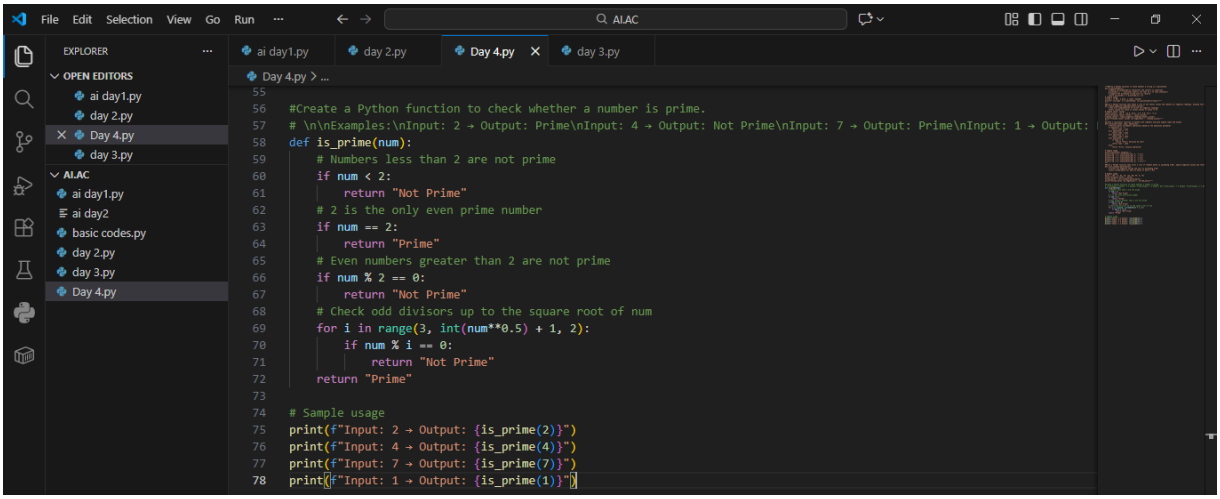
This task demonstrates how **vague prompts cause ambiguous results**. Initially, the AI may not know:

- Sorting order , Data constraints , Output format

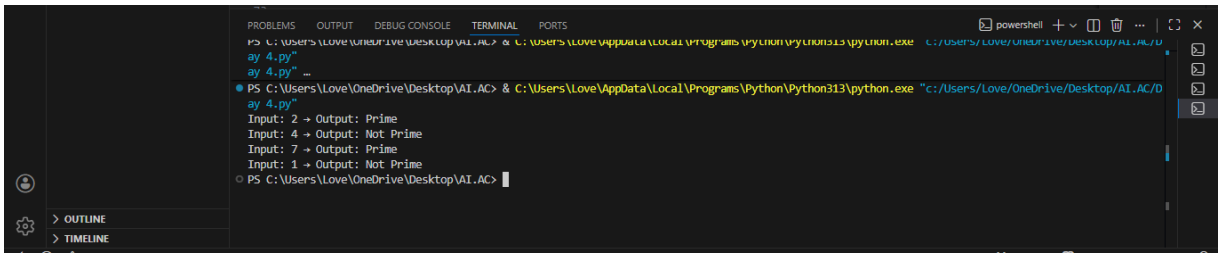
By refining the prompt, we guide the AI to generate **accurate and efficient sorting logic**.
This highlights the importance of **specific instructions in prompt engineering**.

Task 3: Few-Shot Prompting – Prime Number Validation

Prompt: Create a Python function to check whether a number is prime. Examples:\n Input: 2 → Output: Prime\n Input: 4 → Output: Not Prime\n Input: 7 . Use these examples to design the logic



OUTPUT:



EXPLANATION:

Few-shot prompting means providing **example inputs and outputs** along with the prompt.
This helps the AI:

- Understand edge cases , Improve accuracy , Avoid logical mistakes

Compared to a simple prompt, few-shot prompting results in **more reliable prime-checking logic**.

Task 4: Prompt-Guided UI Design – Student Grading System

Prompt : Create a Python program with a simple user interface that accepts student marks, calculates total marks, percentage, and grade, and displays the result.

```python
#Create a Python program with a simple user interface that accepts student marks, calculates total marks, percentage, and g
def calculate_grade(marks):
    # Calculate total marks
    total_marks = sum(marks)
    # Calculate percentage
    percentage = (total_marks / (len(marks) * 100)) * 100
    # Determine grade based on percentage
    if percentage >= 90:
        grade = 'A'
    elif percentage >= 80:
        grade = 'B'
    elif percentage >= 70:
        grade = 'C'
    elif percentage >= 60:
        grade = 'D'
    else:
        grade = 'F'
    return total_marks, percentage, grade
# User interface to accept student marks
def main():
    print("Student Marks Calculator")
    marks = []
    num_subjects = int(input("Enter the number of subjects: "))
    for i in range(num_subjects):
        mark = float(input(f"Enter marks for subject {i + 1} (out of 100): "))
        marks.append(mark)
    total, percentage, grade = calculate_grade(marks)
    print(f"\nTotal Marks: {total}")
    print(f"Percentage: {percentage:.2f}%")
    print(f"Grade: {grade}")
if __name__ == "__main__":
    main()
```

Output:

```
Student Marks Calculator
Enter the number of subjects: 5
Enter marks for subject 1 (out of 100): 50
Enter marks for subject 2 (out of 100): 60
Enter marks for subject 3 (out of 100): 70
Enter marks for subject 4 (out of 100): 90
Enter marks for subject 5 (out of 100): 80

Total Marks: 350.0
Percentage: 70.00%
Grade: C
PS C:\Users\Love\OneDrive\Desktop\AI.AC>
```

Explanation:

This task focuses on using prompts to guide program structure and user interaction.

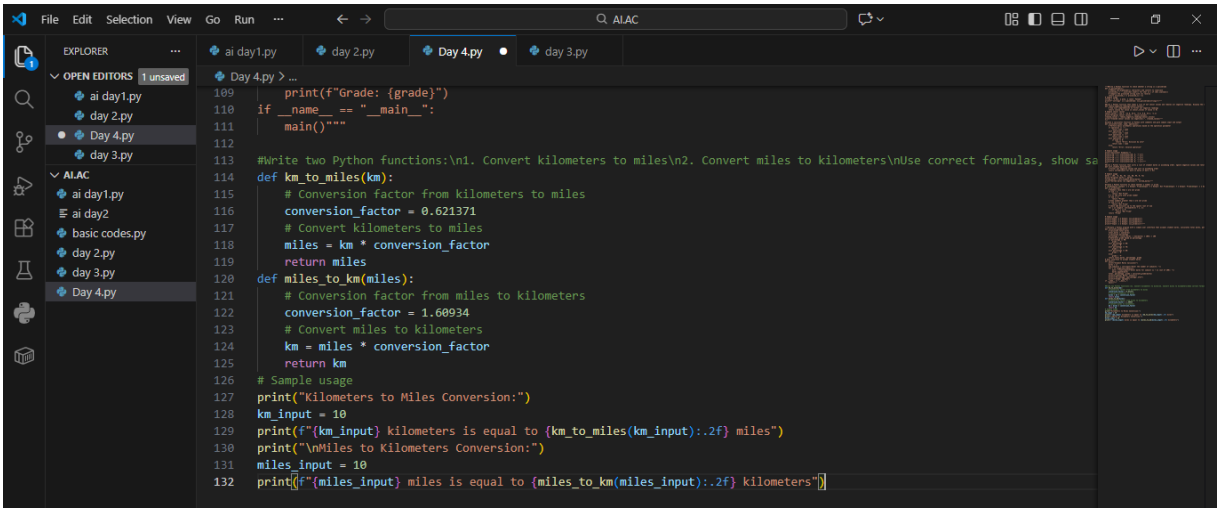
Instead of a graphical UI, a console-based UI is used for:

- Simplicity , Code compatibility , Clear user interaction

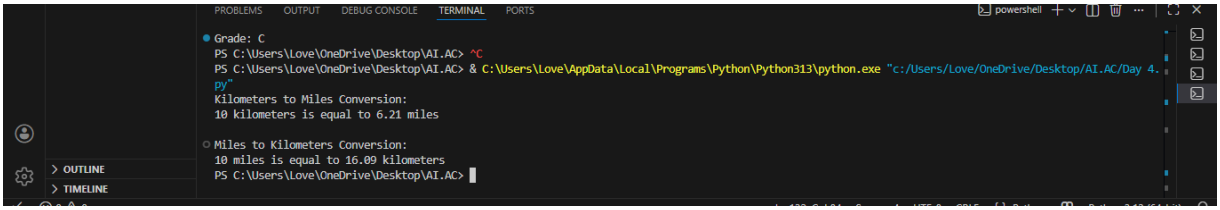
Task 5: Prompt Specificity – Unit Conversion Function

Prompt:

Write two Python functions:\n1. Convert kilometers to miles\n2. Convert miles to kilometers\nUse correct formulas, show sample input/output, and add comments explaining the logic



Output:



Explanation:

This task highlights how clear and specific prompts improve code accuracy.
A vague prompt may produce incomplete or incorrect conversions.

When formulas and requirements are clearly stated, the AI generates:

- Accurate calculations, Reusable functions, Well-documented code

This proves that **prompt specificity directly affects output quality**.